

Consideraciones generales:

- En la clase **Nodo**, los métodos **getIzquierdo()** y **getDerecho()** permiten acceder a los hijos izquierdo y derecho del nodo actual, respectivamente. Estos métodos retornan un objeto **Nodo** o **null** si no hay hijo en esa posición. El método **getValor()** retorna el valor que guarda el **Nodo**
- Alcanza con imprimir el valor de un **Nodo** para considerarlo recorrido

Ejercicios

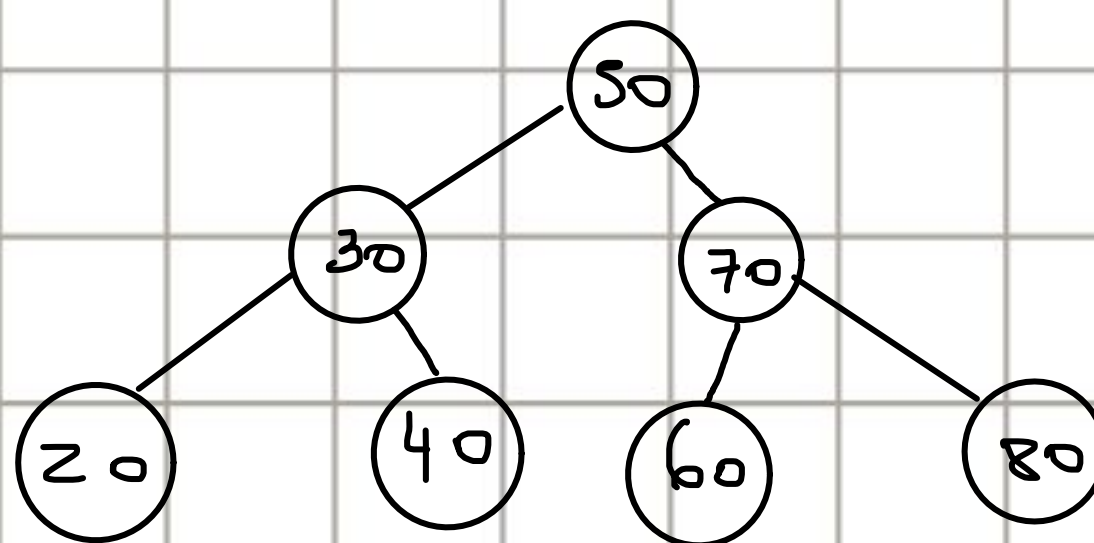
1. ¿Cuál es la salida esperada al ejecutar el método **recorridoInOrden()** después de insertar los elementos {50, 30, 70, 20, 40, 60, 80} en el árbol?

a) 20 30 40 50 60 70 80

b) 50 30 70 20 40 60 80

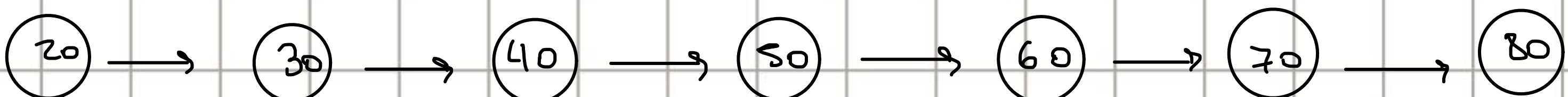
c) 80 70 60 50 40 30 20

d) 20 40 30 60 80 70 50



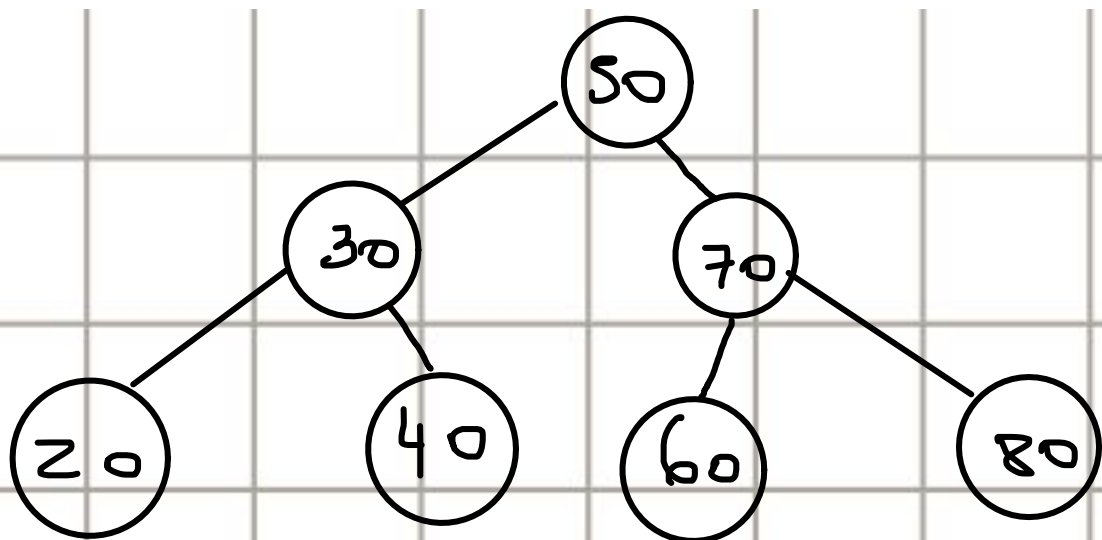
→ INORDER (I R D)

→ Se recorren todos los elementos del **subárbol izquierdo**, luego la raíz y por último todos los elementos del **subárbol derecho**, empezando desde la raíz

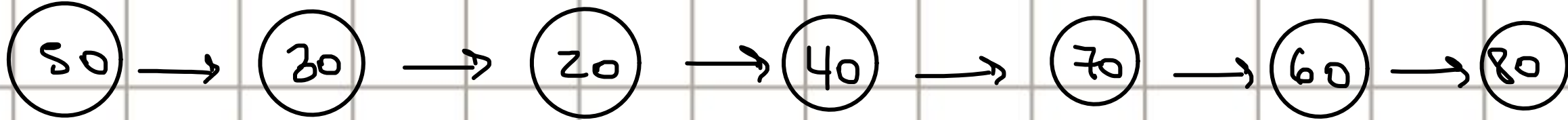


2. Empareja cada tipo de recorrido con su secuencia de salida esperada tras la inserción de los elementos {50, 30, 70, 20, 40, 60, 80}.

a) Inorden	1) 50 30 20 40 70 60 80
b) Preorden	2) 20 30 40 50 60 70 80
c) Postorden	3) 20 40 30 60 80 70 50



→ Preorden (R I D)
(1)



→ Postorden (I D R)



3. Completa el método `recorridoPreOrden()` en la clase `ABB`. La firma del método es:

```
private void recorridoPreOrdenRec(Nodo nodo)
```

```
public class ABB { no usages

    //RID
    private void recorridoPreOrdenRec(NodoT nodo) { 2 usages
        if(nodo != null) {
            System.out.println(nodo.getDato());
            recorridoPreOrdenRec(nodo.getIzq());
            recorridoPreOrdenRec(nodo.getDer());
        }
    }

    private class NodoT<T> { 5 usages
        private T dato; 2 usages
        private NodoT<T> izq; 2 usages
        private NodoT<T> der; 2 usages

        public NodoT(T dato) { no usages
            this.dato = dato;
            this.izq = null;
            this.der = null;
        }

        public NodoT<T> getIzq() { 1 usage
            return izq;
        }

        public NodoT<T> getDer() { 1 usage
            return der;
        }

        public T getDato() { 1 usage
            return dato;
        }
    }
}
```

4. ¿Que ocurre si se intenta eliminar un nodo sin hijos?
- a) El nodo se elimina por completo.
 - b) El valor del nodo se reemplaza, pero el nodo sigue existiendo.
 - c) Se lanza una excepción.
 - d) No ocurre nada.

5. Implementa un método en la clase `ABB` llamado `contarNodos()` que retorne el número total de nodos del árbol.

```
public int contarNodos()
```

```
1 public class ABB { no usages
2
3     private NodoT raiz; 1 usage
4
5     public int contarNodos() { no usages
6         return contarNodosAux(this.raiz);
7     }
8
9     private int contarNodosAux(NodoT nodo) { 3 usages
10        int res = 0;
11        if(nodo != null) res = contarNodosAux(nodo.getIzq()) + 1 + contarNodosAux(nodo.getDer());
12        return res;
13    }
14 }
```

6. Completa el método `recorridoEnAncho()` en la clase `ABB`, utilizando `Queue` (Cola).

```
public void recorridoEnAncho() {
    if (raiz == null) return;
    Queue<Nodo> cola = new LinkedList<>();
    cola.add(raiz);
    while (!cola.isEmpty()) {
        Nodo actual = cola.poll();
        // Completa con el código relevante
    }
}
```

```
import java.util.LinkedList;
import java.util.Queue;

public class ABB { no usages

    private NodoT raiz; 3 usages

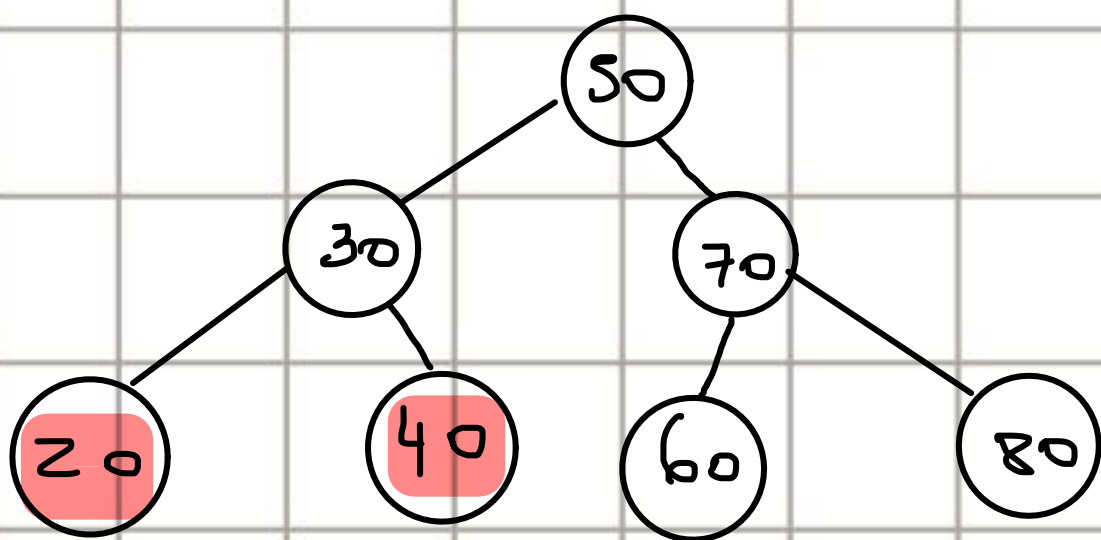
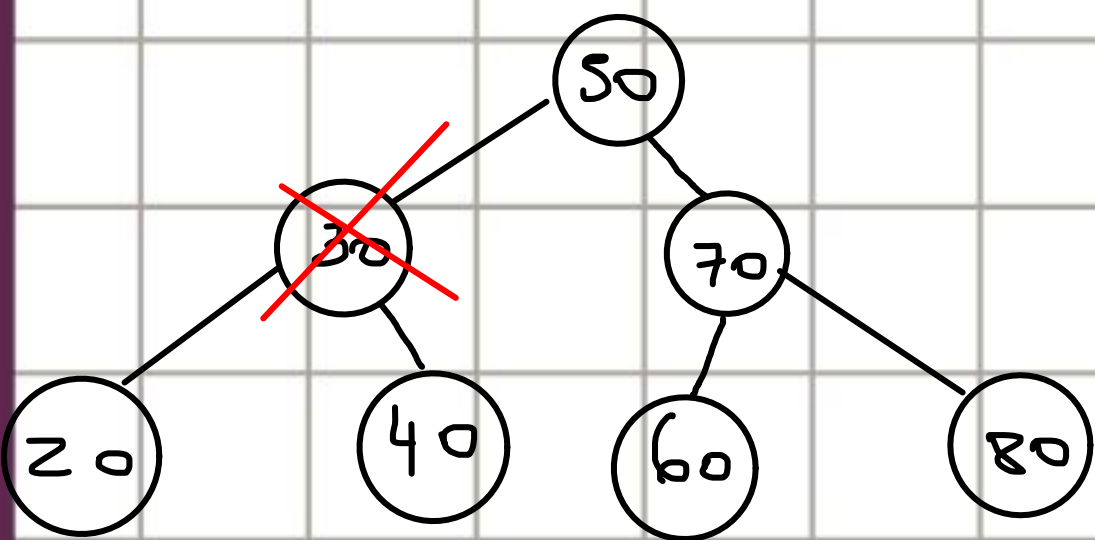
    public void recorridoEnAncho() { no usages
        if (this.raiz == null) return;
        Queue<NodoT> cola = new LinkedList<>();
        cola.add(this.raiz);

        while (!cola.isEmpty()) {
            NodoT actual = cola.poll();

            // Completa con el código relevante
            System.out.println(actual.getDato());

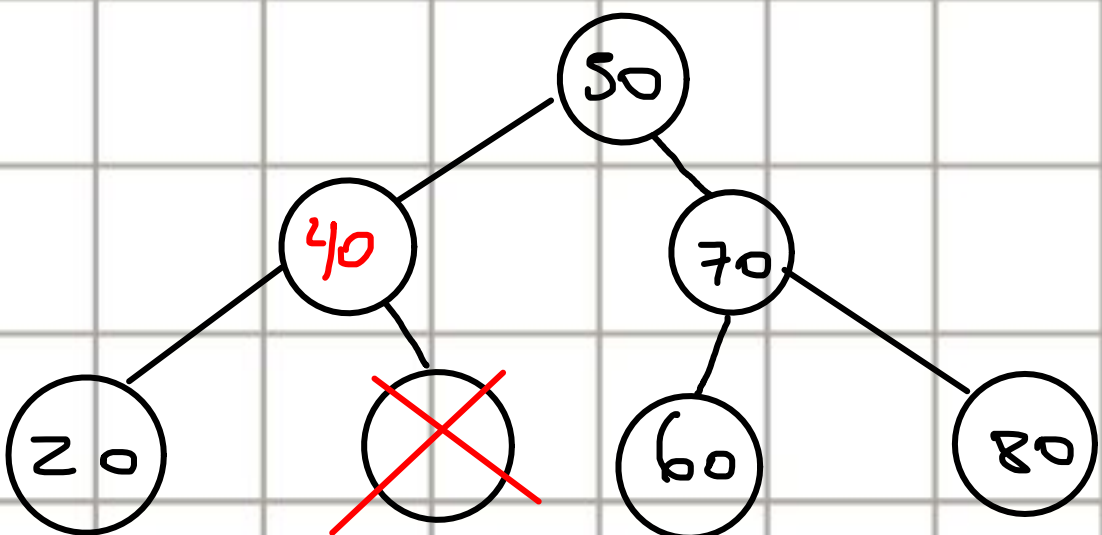
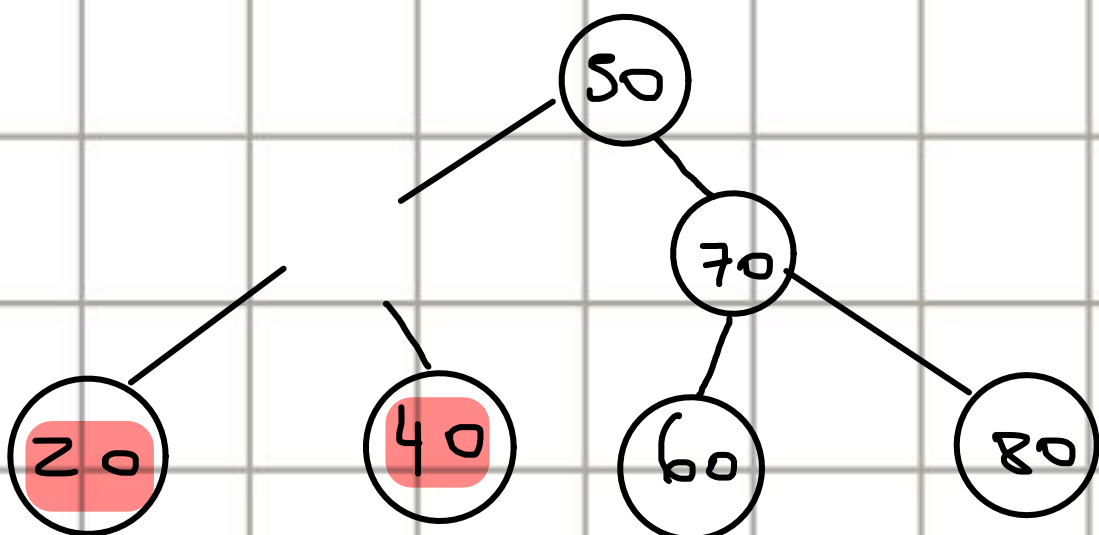
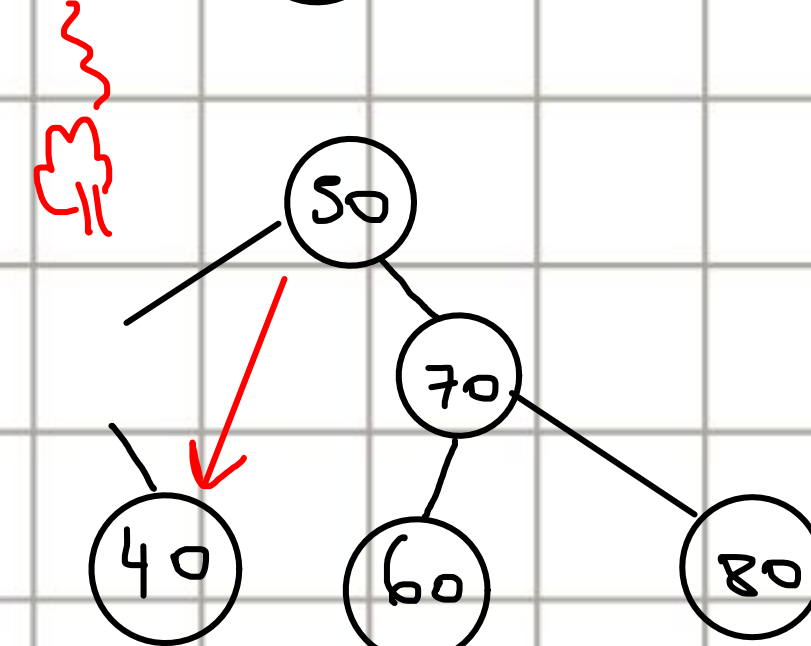
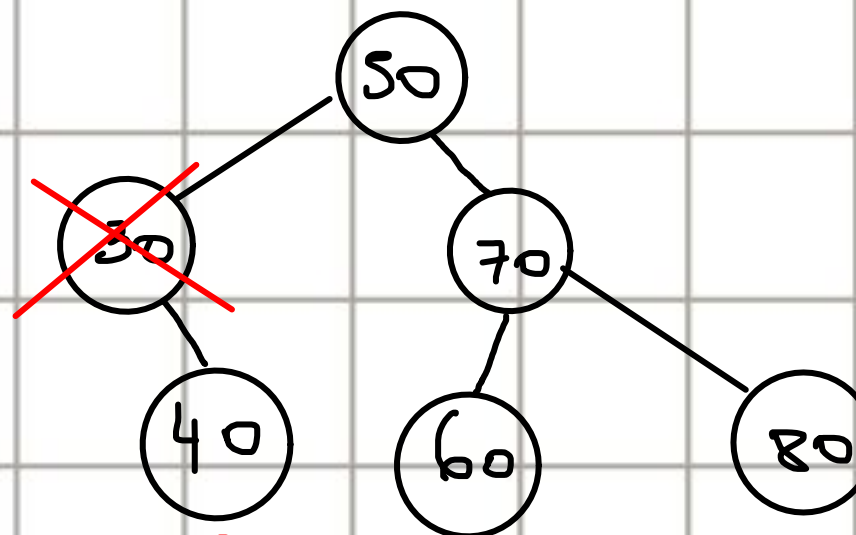
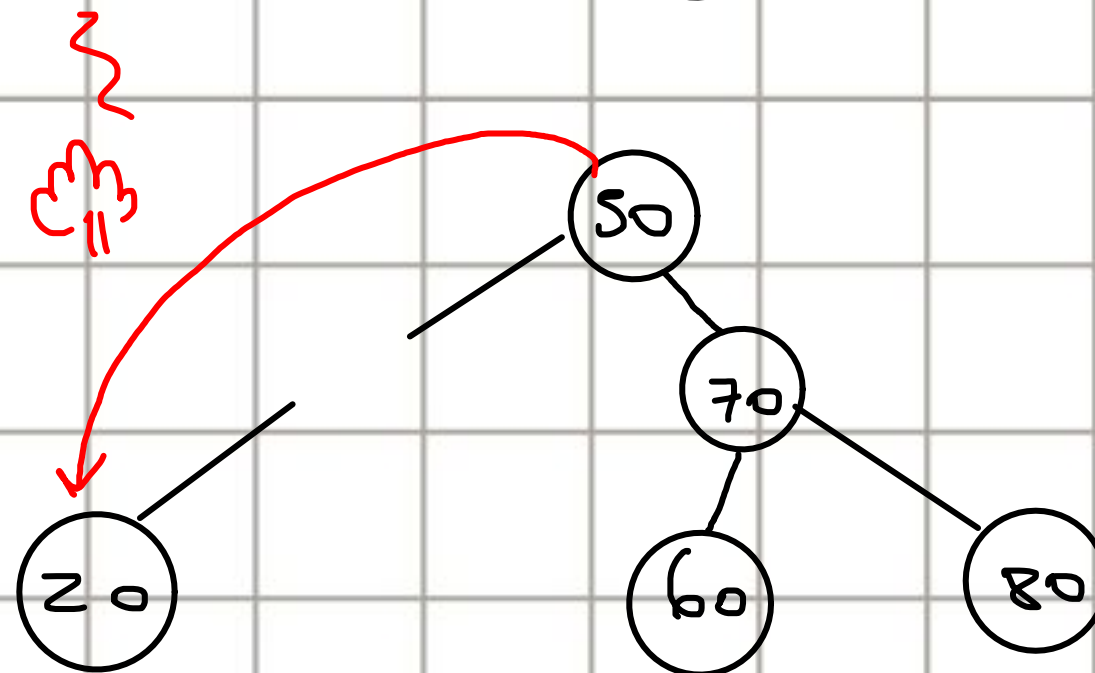
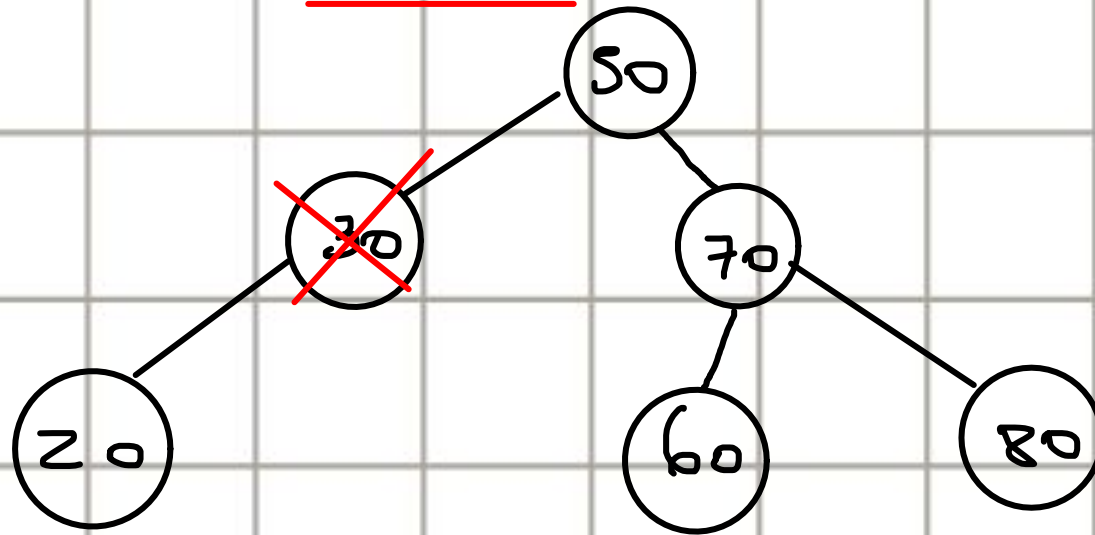
            if(actual.getDer() != null) cola.add(actual.getDer());
            if(actual.getIzq() != null) cola.add(actual.getIzq());
        }
    }
}
```


7. Implementa un método llamado `modificar(int valor, int nuevoValor)` que **remueva el valor de un nodo del árbol e ingrese un nuevo valor**, haciendo uso de los métodos `eliminar(int valor)` e `insertar(int valor)`.

Holaeliminar2 Hijos

IP

IS

1 Hijo


```
public void modificar(int valor, int nuevoValor) {
    eliminar(valor);
    insertar(nuevoValor);
}
```

```
private void eliminar(int valor) { 2 usages
    NodoT aEliminar = buscarNodo(valor);
    if (aEliminar == null) return;

    NodoT padre = buscarPadre(valor);

    if (aEliminar.getIzq() == null && aEliminar.getDer() == null) {
        if (padre == null) this.raiz = null;
        else if (padre.getIzq() == aEliminar) padre.setIzq(null);
        else padre.setDer(null);
    } else if (aEliminar.getIzq() != null && aEliminar.getDer() == null) {
        if (padre == null) this.raiz = aEliminar.getIzq();
        else if (padre.getIzq() == aEliminar) padre.setIzq(aEliminar.getIzq());
        else padre.setDer(aEliminar.getIzq());
    } else if (aEliminar.getIzq() == null && aEliminar.getDer() != null) {
        if (padre == null) this.raiz = aEliminar.getDer();
        else if (padre.getIzq() == aEliminar) padre.setIzq(aEliminar.getDer());
        else padre.setDer(aEliminar.getDer());
    } else {
        NodoT inmediatoSucesor = inmediatoSucesor(aEliminar);
        aEliminar.cambiarDato(immediatoSucesor.getDato());
        eliminar(immediatoSucesor.getDato());
    }
}
```

```
private void insertar(int valor) { 1 usage
    NodoT aInsertar = new NodoT(valor);
    if(this.raiz == null){
        this.raiz = aInsertar;
        return;
    }
    NodoT padre = this.raiz;
    NodoT actual = this.raiz;
    while(actual != null) {
        if(actual.getDato() < valor) {
            padre = actual;
            actual = actual.getDer();
        } else {
            padre = actual;
            actual = actual.getIzq();
        }
    }
    actual = aInsertar;
    if(actual.getDato() < padre.getDato()) padre.setIzq(actual);
    else padre.setDer(actual);
}
```

```
private NodoT buscarNodo(int valor) { 1 usage
    NodoT actual = this.raiz;
    while (actual != null && actual.getDato() != valor){
        if(actual.getDato() < valor) {
            actual = actual.getDer();
        } else {
            actual = actual.getIzq();
        }
    }
    return actual;
}
```

```
private NodoT inmediatoSucesor(NodoT nodo) { 1 usage
    nodo = nodo.getDer();
    while(nodo.getIzq() != null) nodo = nodo.getIzq();
    return nodo;
}
```

```
private NodoT buscarPadre(int valor) { 1 usage
    NodoT actual = this.raiz;
    NodoT padre = this.raiz;
    while (actual != null && actual.getDato() != valor){
        if(actual.getDato() < valor) {
            padre = actual;
            actual = actual.getDer();
        } else {
            padre = actual;
            actual = actual.getIzq();
        }
    }
    if(actual == this.raiz) padre = null;
    return padre;
}
```